

CONTEXT FOUNDRY — INTELLIGENT AUTO-DISCOVERY SYSTEM

The Product That Beats Palantir

Core principle: Upload documents in any domain. The system discovers what they contain, builds its own understanding, answers questions with provenance, and admits what it doesn't know. Zero human ontology engineering. Zero configuration.

ARCHITECTURE: 6 INTELLIGENT COMPONENTS

COMPONENT 1: Document Structure Discovery Engine

Problem: The current system treats all documents as flat text. Tables are chunked like prose. Sections are ignored. This loses structured data.

Solution: Build a StructureAnalyzer that classifies document regions by type:

```
# src/context_foundry/discovery/structure_analyzer.py

class StructureAnalyzer:
    """Analyzes document structure without hardcoded patterns.
    Learns to recognize prose, tables, lists, headers from layout."""

    def analyze(self, raw_text: str) -> List[DocumentRegion]:
        regions = []
        lines = raw_text.split('\n')

        current_region = None
        for i, line in enumerate(lines):
            line_type = self._classify_line(line, i, lines)

            if line_type != current_region:
                if current_region:
                    regions.append(current_region)
                current_region = DocumentRegion(
                    type=line_type, # 'prose', 'table', 'list', 'header', 'code'
                    start_line=i,
                    lines=[]
                )
                current_region.lines.append(line)

            if current_region:
                regions.append(current_region)

        return self._merge_adjacent(regions)

    def _classify_line(self, line: str, index: int, all_lines: List[str]) -> str:
        """Classify a single line by its structural properties.
        No hardcoded regex. Uses statistical properties."""
```

```

stripped = line.strip()

# Table indicators: repeated | characters, consistent column structure
pipe_count = stripped.count('|')
if pipe_count >= 2 and pipe_count / max(len(stripped), 1) > 0.1:
    # Check if surrounding lines also have pipes (table, not inline)
    neighbor_pipes = sum(
        all_lines[j].strip().count('|') >= 2
        for j in range(max(0, index-1), min(len(all_lines), index+2))
        if j != index
    )
    if neighbor_pipes >= 1:
        return 'table'

# Header indicators: short line, followed by blank line or different pattern
if len(stripped) < 100 and stripped and not stripped.endswith('.'):
    if index + 1 < len(all_lines) and all_lines[index + 1].strip() == '':
        return 'header'

# List indicators: starts with bullet, number, or dash
if re.match(r'^[\s]*[\*\^\-\.\d]+[\.\.\)]?\s', stripped):
    return 'list'

# Code indicators: indentation, special characters
if line.startswith('    ') or line.startswith('\t'):
    if any(c in stripped for c in '{}[];='):
        return 'code'

# Default: prose
return 'prose'

def _merge_adjacent(self, regions: List[DocumentRegion]) -> List[DocumentRegion]:
    """Merge adjacent regions of same type."""
    merged = []
    for region in regions:
        if merged and merged[-1].type == region.type:
            merged[-1].lines.extend(region.lines)
            merged[-1].end_line = region.end_line
        else:
            merged.append(region)
    return merged

```

Key difference from hardcoded: The classifier uses statistical properties (pipe density, line length, surrounding context) not a hardcoded regex. It learns to recognize tables in any format.

Integration: The chunker uses StructureAnalyzer output to decide how to chunk each region. Tables are kept intact. Prose is chunked normally. Headers are attached to the following content.

COMPONENT 2: Type Discovery Engine

Problem: The ontology has 256 types but corporate documents use types that aren't in it (PRESIDENT_OF, HAS_BUDGET, DESCRIBES). Humans have to manually add them.

Solution: Build a TypeDiscoveryAgent that observes extraction outputs, clusters unseen relationship patterns, and proposes new types.

```
# src/context_foundry/discovery/type_discovery_agent.py

class TypeDiscoveryAgent:
    """Observes extraction outputs and discovers new entity/relationship types.
    Proposes additions to the ontology based on observed patterns."""

    def __init__(self, session, min_observations: int = 5):
        self.session = session
        self.min_observations = min_observations # Must see pattern N times before proposing
        self.observation_log = [] # All extracted relationship patterns

    def observe_extraction(self, extraction_result: ExtractionResult,
                           document_metadata: DocumentMetadata):
        """Called after every document extraction. Logs observed patterns."""
        for rel in extraction_result.relationships:
            self.observation_log.append(ObservedPattern(
                source_type=rel.source_entity_type,
                target_type=rel.target_entity.target_entity_type,
                relationship_phrase=rel.raw_relationship_type, # What LLM actually said
                source_text=rel.evidence_text,
                document_type=document_metadata.document_type,
                confidence=rel.confidence
            ))

    def discover_types(self) -> List[TypeProposal]:
        """Analyze observations and propose new types for the ontology.
        Called periodically (e.g., after every 10 documents)."""
        proposals = []

        # Cluster 1: Unmapped relationship phrases
        # Find raw relationship phrases that appeared frequently but don't map to known types
        unmapped = self._find_frequent_unmapped()
        for phrase, count, examples in unmapped:
            # Analyze the pattern
            source_types = set(ex.source_type for ex in examples)
            target_types = set(ex.target_type for ex in examples)

            proposal = TypeProposal(
                proposed_type=phrase.upper().replace(' ', '_'),
                source_types=source_types,
                target_types=target_types,
                observation_count=count,
                example_evidence=[ex.source_text for ex in examples[:3]],
                confidence=self._compute_proposal_confidence(examples),
```

```

        proposal_reason=f"Observed {count} times across {len(set(ex.document_type for ex in
        ↪ examples))} document types"
    )
    proposals.append(proposal)

# Cluster 2: Compound role patterns
# "President of NDS" suggests PRESIDENT_OF is a subtype of HOLDS_POSITION
compound_roles = self._find_compound_roles()
for base_role, qualifiers in compound_roles.items():
    proposal = TypeProposal(
        proposed_type=f"{base_role.upper()}_OF",
        source_types={'PERSON'},
        target_types={'ORGANIZATION'},
        observation_count=sum(qualifiers.values()),
        example_evidence=list(qualifiers.keys())[:5],
        confidence=0.85,
        proposal_reason=f"Compound role pattern: '{base_role} of X' observed {len(qualifiers)}
        ↪ times",
        parent_type='HOLDS_POSITION' # Inherits from known type
    )
    proposals.append(proposal)

return proposals

def _find_frequent_unmapped(self) -> List[Tuple[str, int, List[ObservedPattern]]]:
    """Find relationship phrases that appear frequently but aren't in the ontology."""
    from collections import Counter

    # Count all raw relationship phrases
    phrase_counts = Counter(obs.relationship_phrase for obs in self.observation_log)

    # Get known types from ontology
    known_types = set(
        row.relation_type
        for row in self.session.query(OntologyRelation.relation_type).all()
    )

    unmapped = []
    for phrase, count in phrase_counts.most_common():
        if count >= self.min_observations and phrase.upper() not in known_types:
            examples = [obs for obs in self.observation_log
                        if obs.relationship_phrase == phrase]
            unmapped.append((phrase, count, examples))

    return unmapped

def _find_compound_roles(self) -> Dict[str, Counter]:
    """Find patterns like 'President of X', 'VP of Y', 'Director of Z'."""
    compound_pattern = re.compile(
        r'(President|CEO|CFO|CTO|VP|Vice President|Director|Manager|Head|Chief)\s+of\s+(\w+)',
        re.IGNORECASE
    )

```

```

    )

    compounds = {}
    for obs in self.observation_log:
        matches = compound_pattern.findall(obs.source_text)
        for base_role, org in matches:
            if base_role not in compounds:
                compounds[base_role] = Counter()
            compounds[base_role][f"{base_role} of {org}"] += 1

    return compounds

def _compute_proposal_confidence(self, examples: List[ObservedPattern]) -> float:
    """How confident are we that this is a real, distinct type?"""
    # Factors: number of observations, consistency of source/target types,
    # document diversity, extraction confidence
    count = len(examples)
    type_consistency = len(set((ex.source_type, ex.target_type) for ex in examples))
    doc_diversity = len(set(ex.document_type for ex in examples))
    avg_confidence = sum(ex.confidence for ex in examples) / count

    # More observations, more consistent types, more diverse docs = higher confidence
    score = min(0.3 + (count * 0.05) + (doc_diversity * 0.1) + (avg_confidence * 0.3), 0.95)
    return score

```

Integration: After every N documents, TypeDiscoveryAgent runs and proposes new types. A human approver (or auto-approver for high-confidence proposals) adds them to the ontology. Future extractions automatically use the new types.

COMPONENT 3: Cross-Document Validation Engine

Problem: A fact extracted from one document might be wrong. The current system has no way to check if the same fact appears consistently across multiple documents.

Solution: Build a CrossDocumentValidator that tracks facts across documents and validates them through corroboration or contradiction detection.

```

# src/context_foundry/discovery/cross_document_validator.py

class CrossDocumentValidator:
    """Validates facts by checking consistency across multiple documents.
    Facts that appear in multiple documents with consistent details are more reliable."""

    def __init__(self, session):
        self.session = session

    def validate_fact(self, fact: ExtractedFact) -> ValidationResult:
        """Check if this fact is corroborated or contradicted by other documents."""

        # Find all edges between the same entities (or entity pair)

```

```

corroborating = self._find_corroborating_edges(fact)
contradicting = self._find_contradicting_edges(fact)

# Score based on evidence quality
if contradicting:
    # Contradiction found — need to resolve
    return ValidationResult(
        status='CONTRADICTED',
        confidence=self._resolve_contradiction(fact, corroborating, contradicting),
        reasoning=f"Contradicted by {len(contradicting)} sources. Corroborated by
        ↪ {len(corroborating)}.",
        action_required='HUMAN_REVIEW'
    )

if len(corroborating) >= 2:
    return ValidationResult(
        status='CORROBORATED',
        confidence=0.7 + (min(len(corroborating), 5) * 0.05), # More docs = higher confidence
        reasoning=f"Corroborated by {len(corroborating)} independent sources.",
        action_required='AUTO_PROMOTE'
    )

if len(corroborating) == 1:
    return ValidationResult(
        status='SINGLE_SOURCE',
        confidence=0.5,
        reasoning="Only one source found. Needs corroboration.",
        action_required='STAGING'
    )

return ValidationResult(
    status='ORPHAN',
    confidence=0.3,
    reasoning="No corroborating or contradicting evidence found.",
    action_required='STAGING'
)

def _find_corroborating_edges(self, fact: ExtractedFact) -> List[Relationship]:
    """Find edges that support the same fact from different documents."""
    # Same relationship type between same entities
    return self.session.query(Relationship).filter(
        Relationship.source_entity_id == fact.source_entity_id,
        Relationship.target_entity_id == fact.target_entity_id,
        Relationship.relationship_type == fact.relationship_type,
        Relationship.evidence_chunk_id != fact.evidence_chunk_id # Different source
    ).all()

def _find_contradicting_edges(self, fact: ExtractedFact) -> List[Relationship]:
    """Find edges that contradict this fact.

    Contradiction patterns:

```

```

- Two different people hold the same active role
- Same entity has mutually exclusive properties
- Temporal contradictions (start date after end date)
"""

contradictions = []

# Pattern 1: Role conflict
if fact.relationship_type in ('HOLDS_POSITION', 'LEADS', 'MANAGES', 'HEADS', 'CHAIRS'):
    # Find if someone else holds the same role
    role_id = fact.target_entity_id
    other_holders = self.session.query(Relationship).filter(
        Relationship.target_entity_id == role_id,
        Relationship.relationship_type.in(['HOLDS_POSITION', 'LEADS', 'MANAGES', 'HEADS',
        ↪ 'CHAIRS']),
        Relationship.source_entity_id != fact.source_entity_id,
        Relationship.lifecycle_state != 'ARCHIVED'
    ).all()
    contradictions.extend(other_holders)

return contradictions

def _resolve_contradiction(self, fact, corroborating, contradicting) -> float:
    """When facts contradict, determine which is more reliable."""
    # Factors: document authority, evidence explicitness, temporal recency, number of corroborations

    fact_score = self._compute_reliability_score(fact, corroborating)

    best_alt_score = 0
    for alt in contradicting:
        alt_corroborating = self._find_corroborating_edges(alt)
        alt_score = self._compute_reliability_score(alt, alt_corroborating)
        best_alt_score = max(best_alt_score, alt_score)

    if fact_score > best_alt_score + 0.2:
        return fact_score # Our fact is significantly more reliable
    elif best_alt_score > fact_score + 0.2:
        return 0.2 # Our fact is probably wrong
    else:
        return 0.5 # Unclear — needs human review

def _compute_reliability_score(self, fact, corroborating) -> float:
    """Score how reliable a fact is based on its evidence."""
    score = 0.5 # Base

    # More corroborating sources = higher score
    score += min(len(corroborating) * 0.1, 0.3)

    # Explicit language in source text ("appointed as" vs "the CEO, Bob")
    explicit_indicators = ['appointed', 'named', 'elected', 'hired', 'succeeded']
    if any(ind in fact.evidence_text.lower() for ind in explicit_indicators):
        score += 0.15

```

```

# Higher confidence from extraction
score += fact.confidence * 0.1

return min(score, 0.95)

```

COMPONENT 4: Reasoning Engine

Problem: The system finds facts but doesn't reason about them. "Victoria Chen is CEO" and "The CEO announced Q4 earnings" should combine to infer "Victoria Chen announced Q4 earnings."

Solution: Build a LogicalReasoningEngine that applies inference rules to derive new facts from existing ones.

```

# src/context_foundry/discovery/reasoning_engine.py

class LogicalReasoningEngine:
    """Applies logical inference rules to derive new facts from existing graph data."""

    INFERENCE_RULES = [
        # Rule 1: Transitive reporting
        # If A reports_to B and B reports_to C, then A reports_to C (indirectly)
        {
            'name': 'transitive_reporting',
            'pattern': [('A', 'REPORTS_TO', 'B'), ('B', 'REPORTS_TO', 'C')],
            'derive': ('A', 'REPORTS_TO_INDIRECT', 'C'),
            'confidence_multiplier': 0.8
        },
        # Rule 2: Role inference
        # If A holds_position CEO and CEO leads Org, then A leads Org
        {
            'name': 'role_infers_leadership',
            'pattern': [('A', 'HOLDS_POSITION', 'CEO'), ('CEO', 'LEADS', 'Org')],
            'derive': ('A', 'LEADS', 'Org'),
            'confidence_multiplier': 0.85
        },
        # Rule 3: Supply chain
        # If A supplies B and B supplies C, then A indirectly supplies C
        {
            'name': 'transitive_supply',
            'pattern': [('A', 'SUPPLIES', 'B'), ('B', 'SUPPLIES', 'C')],
            'derive': ('A', 'SUPPLIES_INDIRECT', 'C'),
            'confidence_multiplier': 0.7
        },
        # Rule 4: Org membership
        # If A works_at Division and Division is_part_of Org, then A works_at Org
        {
            'name': 'org_hierarchy',
            'pattern': [('A', 'WORKS_AT', 'Division'), ('Division', 'PART_OF', 'Org')],
            'derive': ('A', 'WORKS_AT', 'Org'),

```



```

        'confidence_multiplier': 0.9
    },
]

def __init__(self, session):
    self.session = session

def infer(self, max_hops: int = 2) -> List[InferredFact]:
    """Run all inference rules against the current graph.
    Returns newly inferred facts that don't already exist."""
    inferred = []

    for rule in self.INFERENCE_RULES:
        matches = self._find_pattern_matches(rule['pattern'])
        for match in matches:
            derived = self._apply_rule(rule, match)
            if derived and not self._fact_exists(derived):
                inferred.append(derived)

    return inferred

def _find_pattern_matches(self, pattern: List[Tuple]) -> List[Dict]:
    """Find all instances in the graph that match the rule pattern."""
    # This is a graph pattern matching problem
    # For simplicity, implement as recursive traversal
    matches = []

    # Start with the first edge in the pattern
    first_edge = pattern[0]
    candidates = self.session.query(Relationship).filter(
        Relationship.relationship_type == first_edge[1]
    ).all()

    for candidate in candidates:
        match = {first_edge[0]: candidate.source_entity_id,
                 first_edge[2]: candidate.target_entity_id}

        # Try to match remaining edges
        if self._match_remaining(pattern[1:], match):
            matches.append(match)

    return matches

def _match_remaining(self, remaining_pattern, current_match) -> bool:
    """Recursively match remaining edges in the pattern."""
    if not remaining_pattern:
        return True

    next_edge = remaining_pattern[0]
    source_var, rel_type, target_var = next_edge

```

```

# Resolve variables to actual entity IDs
source_id = current_match.get(source_var, source_var)
target_id = current_match.get(target_var, target_var)

# If both are resolved, check if edge exists
if source_id in current_match and target_id in current_match:
    edge = self.session.query(Relationship).filter(
        Relationship.source_entity_id == current_match[source_id],
        Relationship.relationship_type == rel_type,
        Relationship.target_entity_id == current_match[target_id]
    ).first()
    return edge is not None and self._match_remaining(remaining_pattern[1:], current_match)

# If one is unresolved, find matching edges
if source_id in current_match:
    candidates = self.session.query(Relationship).filter(
        Relationship.source_entity_id == current_match[source_id],
        Relationship.relationship_type == rel_type
    ).all()
    for candidate in candidates:
        current_match[target_var] = candidate.target_entity_id
        if self._match_remaining(remaining_pattern[1:], current_match):
            return True
        del current_match[target_var]

return False

```

COMPONENT 5: Confidence Scoring via Logical Inference

Problem: Confidence is based on document proximity and source counts, not logical reasoning.

Solution: Replace numeric thresholds with the LogicalInferenceEngine. Confidence is determined by:

- Can the fact be logically inferred from other facts? (high confidence)
- Is it corroborated across documents? (medium-high)
- Is it a single-source extraction? (medium)
- Is it contradicted? (low, flagged for review)

```

# In the hybrid orchestrator:

def compute_confidence(fact, graph_context):
    # Step 1: Check for contradictions (always check first)
    contradictions = logical_inference_engine.find_contradictions(fact)
    if contradictions:
        return Confidence.CONTRADICTED, contradictions

    # Step 2: Check if logically inferable
    inferences = logical_inference_engine.infer_from(fact)
    if inferences:

```

```

        return Confidence.HIGH, inferences # Can be proven from other facts

# Step 3: Check cross-document corroboration
corroboration = cross_document_validator.validate_fact(fact)
if corroboration.status == 'CORROBORATED':
    return Confidence.HIGH, corroboration

# Step 4: Check structural completeness
gaps = structural_completeness_check(fact)
if gaps:
    return Confidence.MEDIUM, gaps # Fact exists but gaps remain

# Step 5: Single source
return Confidence.MEDIUM if fact.extraction_confidence > 0.7 else Confidence.LOW

```

COMPONENT 6: Domain Adaptation Layer

Problem: The system works for corporate documents but needs manual changes for maritime, healthcare, etc.

Solution: Build a DomainDetector that identifies the domain of incoming documents and automatically suggests relevant types from the ontology's domain templates.

```

# src/context_foundry/discovery/domain_detector.py

class DomainDetector:
    """Detects the domain of documents and adapts the extraction pipeline."""

    def detect(self, documents: List[Document]) -> DomainProfile:
        """Analyze a batch of documents to determine their domain."""

        # Extract key terms from all documents
        all_terms = Counter()
        for doc in documents:
            terms = self._extract_domain_terms(doc.content)
            all_terms.update(terms)

        # Compare against domain signatures
        domain_scores = {}
        for domain_name, signature in self._load_domain_signatures().items():
            score = self._compute_domain_match(all_terms, signature)
            domain_scores[domain_name] = score

        # Return best match
        best_domain = max(domain_scores, key=domain_scores.get)
        return DomainProfile(
            domain=best_domain,
            confidence=domain_scores[best_domain],
            candidate_types=self._get_types_for_domain(best_domain)
        )

```

```

def _extract_domain_terms(self, text: str) -> Counter:
    """Extract terms that indicate domain."""
    # Use TF-IDF-like scoring: terms that appear frequently in this doc
    # but are rare in general English
    words = re.findall(r'\b[A-Z][a-z]+(?:\s+[A-Z][a-z]+)*\b', text)
    return Counter(words)

def _load_domain_signatures(self) -> Dict[str, Counter]:
    """Load domain-specific term signatures from the ontology."""
    # These are learned over time, not hardcoded
    # Initially seeded from domain_template tables in the ontology
    signatures = {}
    for template in self.session.query(DomainTemplate).all():
        signatures[template.domain_name] = Counter(template.signature_terms)
    return signatures

```

WHAT REPLIT SHOULD BUILD (Priority Order)

Week 1: Structure + Type Discovery

1. **StructureAnalyzer** (Component 1) — handles tables, sections, lists without hardcoded regex
2. **TypeDiscoveryAgent** (Component 2) — observes extractions, proposes new types automatically
3. **Integration:** Wire both into the extraction pipeline so documents are analyzed for structure before extraction, and new types are discovered and added to the ontology automatically

Week 2: Validation + Reasoning

4. **CrossDocumentValidator** (Component 3) — validates facts across multiple documents
5. **LogicalReasoningEngine** (Component 4) — applies inference rules to derive new facts
6. **Integration:** Replace numeric confidence with logical inference-based confidence

Week 3: Domain Adaptation + Polish

7. **DomainDetector** (Component 5) — auto-detects document domains
8. **Integration:** Full end-to-end test with multiple domain document sets

WHAT THIS SYSTEM DOES THAT PALANTIR CANNOT

Capability	Context Foundry (Intelligent)	Palantir Foundry
Ontology creation	Auto-discovers from documents	Human consultants, 3-6 months
Table handling	Learns document structure	Manual schema mapping
Type expansion	Proposes new types from observations	Change request, engineering review
Cross-document validation	Automatic corroboration/contradiction	Manual data quality checks

Table 1 – continued

Capability	Context Foundry (Intelligent)	Palantir Foundry
Logical inference	Derives new facts from existing	Not available
Domain adaptation	Auto-detects and adapts	New deployment, new ontology
Cost per domain	Near zero (automatic)	\$200K-500K professional services
Time to first insight	Hours	6-18 months

This is the product. Not 90/100. A system that learns what you teach it and thinks about what it knows.

STOPPING CONDITION

Build Component 1 (StructureAnalyzer) first. Test it on 5 documents with tables. If it correctly identifies and preserves table structure without hardcoded regex, the architecture is sound. Build the rest.

If it doesn't work — if tables are still split or misidentified — debug the approach before building Components 2-6.

Start with Component 1. Measure. Then Component 2. Measure. No more building everything at once.